

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

2026-09-21

Rules, judge, loop

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

Timing, dividers included: open + check-in 5 · git 7 · from a model to an agent 18 · three standards 15 · live kick-off 10 · your turn 30 · retro 4 · wrap 1 = 90, no slack. Per-slide weights are in the plan, not on the slides. The GPT-2 block puts the lecture four minutes over: cut the history slides (12–14, NVIDIA included) by default, run them only if ahead; the GPT-2 block (15–19) is the next cut. TA-run. Slides carry ideas only: every command is on the Week 4 page, open on their laptops, and in these notes. Presenting laptop: SplitIt with feature 1 merged, clean main, Codex open, GitHub Desktop signed in, data reset.

Today.

- 1 Check-in
- 2 Git: one change, start to finish
- 3 From a model to an agent
- 4 Three standards
- 5 Build one, live — then you build one
- 6 Retro: one new rule

2026-09-21

Rules, judge, loop

└ Today.

Three pictures carry the first half, and each one is built up over several slides rather than shown at once: the git graph, the model-to-agent stack, and the ReAct loop. Say so: "you will see the same picture grow." That is the whole visual method of the class.

CLASS 4

Today.

- 1 Check-in
- 2 Git: one change, start to finish
- 3 From a model to an agent
- 4 Three standards
- 5 Build one, live — then you build one
- 6 Retro: one new rule

Your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.

2026-09-21

Rules, judge, loop

└─ Your GitHub page.

Walk the room looking at browsers, not code. Thirty seconds a row; note who is behind, move on. They can pull main and run the feature-1 tests while you walk; the command is on the Week 4 page. Don't debug git at the front: the next six slides are the explanation, and the page has the "when git is confusing" table from last week.

CHECK-IN · 5 MIN

Your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.



HOW A TEAM STAYS SANE

One change, start to finish.

`main` is the version everyone trusts. Here is how a change gets into it.

2026-09-21

Rules, judge, loop



HOW A TEAM STAYS SANE

One change, start to finish.

`main` is the version everyone trusts. Here is how a change gets into it.

Six minutes for the whole story, about fifty seconds a slide. Five of the six are the previous picture plus one move, so do not narrate them twice — point at what is new and go on. They met repo, commit, branch and push last week as words; this is the same words as a single narrative, in the shape they will actually see in GitHub Desktop. Frame it the way they already think: a draft, checkpoints, a review, a sign-off.

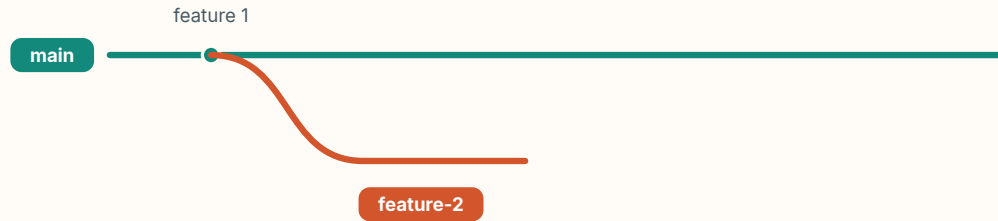
main



One version, one history, one truth. No folder full of *model_v3_FINAL_jm*.



A branch.

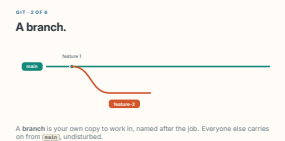


A **branch** is your own copy to work in, named after the job. Everyone else carries on from `main`, undisturbed.

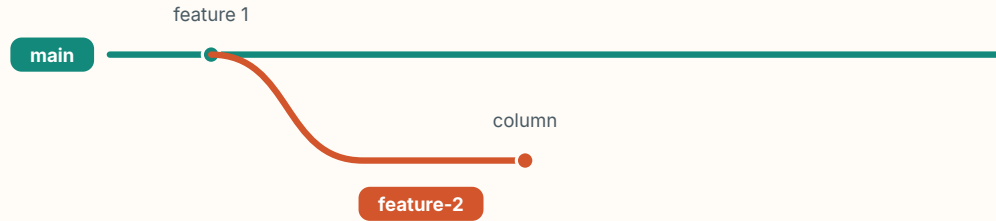
2026-09-21

Rules, judge, loop

└─ A branch.



A commit.

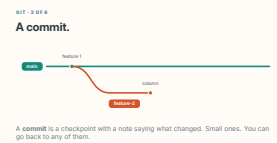


A **commit** is a checkpoint with a note saying what changed. Small ones. You can go back to any of them.

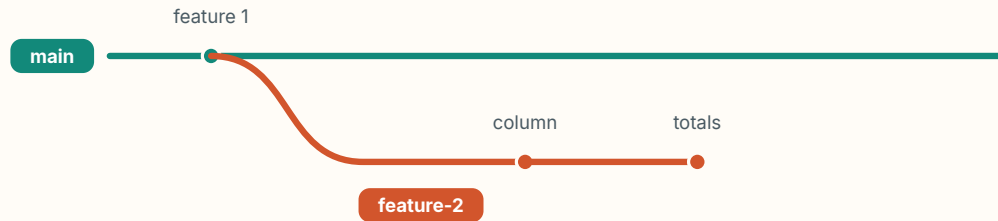
2026-09-21

Rules, judge, loop

└─ A commit.



Working on a branch.



Two commits in and half of it broken. Everyone else's copy is untouched.

2026-09-21

Rules, judge, loop

└ Working on a branch.

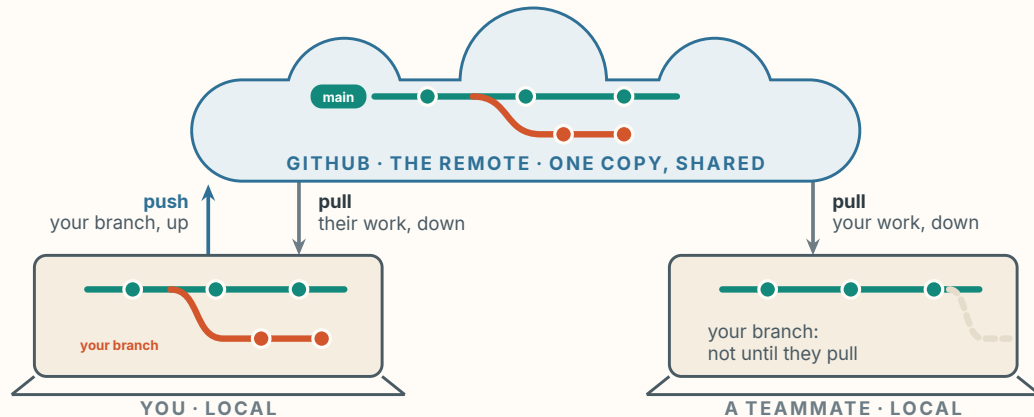
GIT · 4 OF 6

Working on a branch.



Two commits in and half of it broken. Everyone else's copy is untouched.

Local, and remote.

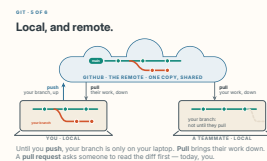


Until you **push**, your branch is only on your laptop. **Pull** brings their work down. A **pull request** asks someone to read the diff first — today, you.

2026-09-21

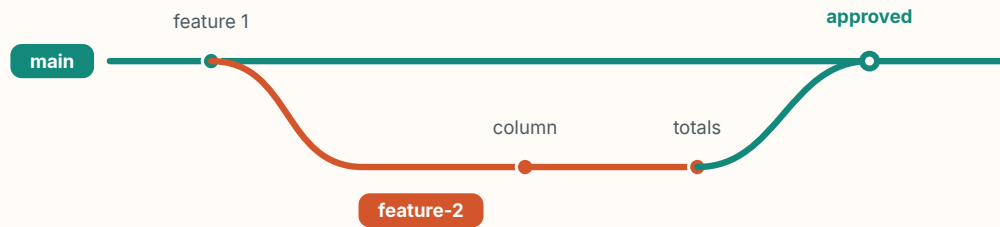
Rules, judge, loop

Local, and remote.



The only slide in the act that leaves the graph, and the one that fixes the vocabulary: *local* is the copy on a laptop — yours, or anyone's — and *remote* is the one copy on GitHub that all of them talk to. Walk it: your branch is on your screen. Push, and it is in the cloud too. Look at the teammate's screen: not there — and it will not be until they pull. That is the whole meaning of "remote": one shared copy, and everything else is a laptop that has to ask. The three moves are the only things that cross the gap: push up, pull down, and a pull request, which is asking someone to read the diff before the cloud's main takes it. If someone asks where "add"/"staging" went: GitHub Desktop turned it into the checkboxes next to each changed file, so they have already used it without the name.

Merge.



Merge is the sign-off. **main** now has your work, and the next person starts from there.

2026-09-21

Rules, judge, loop

└─ Merge.

GIT · 6 OF 6

Merge.



Merge is the sign-off. **main** now has your work, and the next person starts from there.

Land the cycle: they are back at slide one, with a bigger main. That is the habit. Two rules fall out of the story rather than being announced: name the branch after the job (slide 2), and keep commits small enough to explain (slide 3). If someone asks what happens to the branch: it is finished, GitHub offers to delete it, say yes. Then turn: "the thing that will make those commits today is not you. So: what is it?"

agent

FROM A CHAT MODEL TO AN AGENT

What is actually running.

A model that can only write text — and the software around it that lets it act.

2026-09-21

Rules, judge, loop

agent

FROM A CHAT MODEL TO AN AGENT

What is actually running.

A model that can only write text — and the software around it that lets it act.

Sixteen minutes. Two history slides first, so the thing feels like engineering rather than magic; then the stack is built in four moves; then one tool call in full; then reasoning, watched; then the loop gets its name; then what it can hold. Every slide in the act is one picture and one sentence. If you are behind, the two history slides are the ones to cut.

The long run-up.



The ideas were all there. What was missing arrived after 2007: **cheap parallel arithmetic** (graphics cards) and **the web as text**.

NVIDIA CUDA (2007) · Krizhevsky, Sutskever & Hinton, AlexNet (2012) · Common Crawl (public crawls from 2011)

2026-09-21

Rules, judge, loop

— The long run-up.



Ninety seconds, left to right. The dates are on the slide; the names and the story are here. 1958, Rosenblatt's perceptron: a machine that adjusts its own weights from examples instead of being programmed rule by rule. That is still the whole idea. 1969, Minsky and Papert's book *Perceptrons*: one layer can only separate what a straight line separates, so it cannot learn XOR. Funding collapsed; the first AI winter. Careful with the attribution: they wrote the critique that made multi-layer networks the open problem; they did not invent them. 1986, Rumelhart, Hinton and Williams in *Nature*: backpropagation sends the error back through every layer, so many layers finally become trainable. The answer to 1969, seventeen years late. 1997, Hochreiter and Schmidhuber: LSTM carries memory across a sequence. 2017, Vaswani et al.: the Transformer drops recurrence for attention and trains in parallel. Then the payoff: none of this is new. What arrived was compute — CUDA in 2007 let ordinary programmers run their maths on a graphics card, and in 2012 AlexNet trained on two of them and cut the ImageNet error by nearly half (15% against 26%) — and data: Common Crawl, a non-profit giving the public web away as text. Neither is an idea about intelligence. If someone asks why this happened now and not in 1990, that is the answer. If it comes up: Hinton shared the 2024 Nobel Prize in Physics with Hopfield for this line of work.

Why NVIDIA?



Why is NVIDIA the most valuable company in the world?

One piece of software.

A CPU



a few, fast,
one thing each

A GPU



thousands, slower,
all at once

GAMES · 1999

Millions of pixels,
none waiting for
another. Parallel by
necessity.

CUDA · 2007

Any maths, by any
programmer, on the
games chip.

MINING · 2010s

The same trick:
billions of hashes,
side by side.

TRAINING · 2012 →

AlexNet on two
gaming cards.
Today: thousands
of chips, one model.

The chip was built for games. CUDA let everyone else use it — and training a neural network is nothing but parallel arithmetic.

GeForce 256 (1999) · CUDA 1.0 (2007) · Krizhevsky et al. (2012), two GTX 580s · first company past \$4 trillion, July 2025 · logo © NVIDIA

2026-09-21

Rules, judge, loop

Why NVIDIA?

A minute; it is an aside, but it is the one company name in the room everyone knows, and the answer is a software story, which is the run-up slide's point. Ask the question and take a guess or two (chips, AI). Then the answer: CUDA, 2007. A graphics card draws millions of pixels at once, and no pixel waits for another, so the chip is thousands of small slow workers instead of a few fast ones. That is the top-right picture. Until 2007 you could only use it by pretending your problem was a picture. CUDA let any programmer write ordinary maths for it. Then the same parallel trick found its uses in order: games, then crypto mining (billions of hashes, side by side: the 2017 and 2021 card shortages), then in 2012 AlexNet was trained on two gaming cards and cut the ImageNet error by nearly half, and today a language model is trained on tens of thousands of these chips at once, because training is nothing but multiplying big grids of numbers. The valuation: NVIDIA passed four trillion dollars in July 2025, the first company to; say "as of this year" and check the number the week you teach. The lesson to carry: the ideas were old, the compute arrived, and the thing that unlocked the compute was a piece of software.

HOW WE GOT HERE · AN ASIDE · THE ANSWER IS CUDA

Why NVIDIA?



2018 onwards.



The job changed twice: when it learned to take **instructions**, and when it got **tools and a loop**. The next four slides are that second change.

Bottom row, parameters: Radford et al. (2018, 2019), Brown et al. (2020) · GPT-4: an unconfirmed 2023 report (SemiAnalysis) · none published since

2026-09-21

Rules, judge, loop

└─ 2018 onwards.



Two minutes, one sweep left to right. Say the four band names as you go (completing, conversing, acting, persisting). That is the shape to remember, not the model names. The last sentence on the slide is the hand-off: everything from 2023 on is "tools and a loop", and the next four slides build exactly that. Dates are approximate release months on purpose. The bottom row is the size story, and it ends: GPT-1 had 117 million parameters, GPT-2 1.5 billion, GPT-3 175 billion, a thousandfold in two years, and then the numbers stop. OpenAI did not publish GPT-4's; the 1.8 trillion figure is a 2023 leak (a mixture of experts, so not all of it runs per token) and nobody has confirmed it. Nothing since has a public count. If someone asks why: size stopped being the whole story around 2022, and what changed after that is on the right of the timeline, not the bottom.

Ovid's Unicorn.

A HUMAN WROTE

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

GPT-2 CONTINUED

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. [...] Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans.

2026-09-21

Rules, judge, loop

└ Ovid's Unicorn.

One minute, and read it aloud; the room will laugh at the right place. This is the sample OpenAI published with GPT-2 in February 2019, the one that made the field sit up: a model that only predicts the next word, given three sentences, keeps a news story going with a named scientist, a university, and quotes. Two things to plant. First, "completing": nobody asked it a question, it continued the text, which is all a language model does and is the top-left band on the timeline. Second, the four horns: two sentences in, a unicorn has four of them, and the prose never flinches. Fluency and truth are different properties, and only one of them is what the model is trained for. That is why Act 3 has a judge. Say it was the best of ten tries; the paper says so.

Ovid's Unicorn.

A HUMAN WROTE

In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

GPT-2 CONTINUED

The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. [...] Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans.

Guess the next word.

A PAGE FROM THE WEB

The scientist named the population, after their distinctive _____

THE PAGES

WebText 8 million pages 40 GB

every page a Reddit user had linked to, in 2017. GPT-3, a year later, read Common Crawl: a filtered copy of the public web.

That is all training is: hide the next word, guess, get corrected. No grammar lesson, no facts typed in — just 1.5 billion numbers, nudged a few billion times.

Radford et al. (2019): WebText, 8 million pages, 40 GB · Brown et al. (2020): GPT-3 on Common Crawl, filtered · percentages illustrative

ITS GUESS

horn 62%
colour 14%
feature 9%
size 5%
name 3%

off by a bit?
nudge the numbers

then the next word,
a few billion times over

the page says: horn

2026-09-21

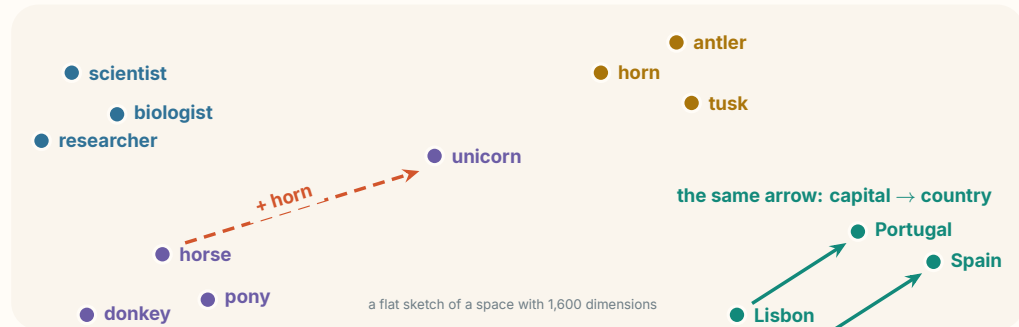
Rules, judge, loop

Guess the next word.

A minute. Walk the picture left to right, then down. A page from the web with the next word covered. The model guesses: a whole list, each with a probability. The page is uncovered: horn. If the model put too little weight on horn, every one of its numbers is nudged a hair in the direction that would have made horn more likely. Then the next word, on the next page, for weeks. Nobody typed in a fact or a grammar rule; everything it knows fell out of that game. The data: OpenAI did not use Common Crawl for GPT-2. They said it was too messy and built WebText: every outbound link from a Reddit post with at least three upvotes, 45 million links, 8 million pages after cleaning, 40 GB of text. Common Crawl, the non-profit copy of the public web from slide 12, is what GPT-3 was trained on a year later, after filtering. The numbers on the bars are made up to look right; the real list has 50,257 entries, one per token, and a token is often a piece of a word rather than a whole one.



Words become places.



Encode: a word becomes a position. Decode: a position becomes the nearest word. Neighbours mean similar things; directions mean relations.

Positions illustrative · arrow arithmetic: Mikolov et al. (2013), king — man + woman ≈ queen

2026-09-21

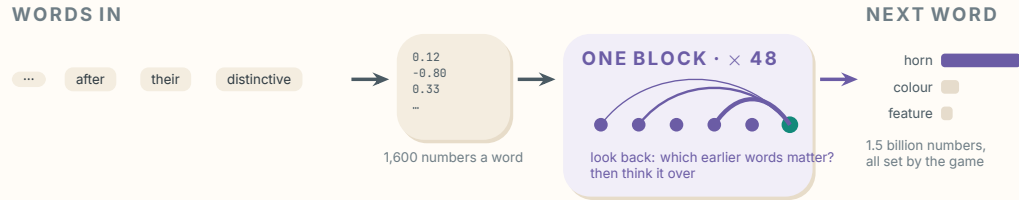
Rules, judge, loop

Words become places.



A minute and a half; this is the slide that makes the unicorn make sense. Every word the model knows is a point, and a point is just a list of 1,600 numbers. The guessing game moved the points. Words that show up in the same places on the web ended up near each other, because the model was rewarded for treating them alike: horse, pony, donkey in one cluster, scientist and biologist in another. Two things to point at. First, the dashed arrow: start at horse, move in the direction that horn lives, and you arrive near unicorn. The model never memorised a definition of unicorn; it has a position for it, and the position is roughly horse plus horn. Second, the teal arrows: Lisbon to Portugal and Madrid to Spain are the same arrow. Relations are directions. That is the famous king minus man plus woman is queen, and it is why the story could name a scientist, give him a university in La Paz and put him in the Andes: each of those is the nearest word to a position the story had moved to. Now the honest half: encode and decode by nearness is also how you get four-horned unicorns. Numbers of horns live near horned, and nothing in the map knows that one is the right number. Say the caveat once: a flat sketch of 1,600 dimensions, and the positions here are drawn by hand to make the point.

Inside the box.

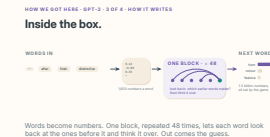


Words become numbers. One block, repeated 48 times, lets each word look back at the ones before it and think it over. Out comes the guess.

2026-09-21

Rules, judge, loop

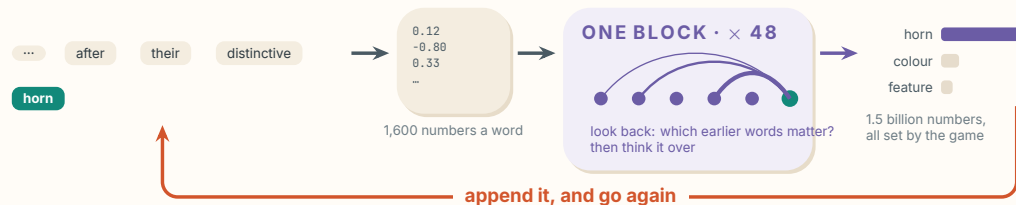
└ Inside the box.



A minute. Left to right. The words go in and become their positions from the last slide. Then one block, the same block, 48 times in a row: each word looks back at the earlier words and decides which ones matter (the arcs: the last word attends strongly to "distinctive", less to "their"), then a small network thinks over what it collected. That look-back is attention, the thing the 2017 Transformer from slide 12 introduced, and it is the whole architecture: nothing else in there. At the top, a probability for every word in the vocabulary. GPT-2's 1.5 billion numbers are the weights inside those 48 blocks plus the word positions, and every one of them was set by the guessing game. Do not go deeper than this; nobody in the room needs the maths, they need the shape: numbers in, look back, guess.

One word at a time.

WORDS IN



SO THE STORY GROWS

...distinctive horn · These four -horned , silver -white unicorns

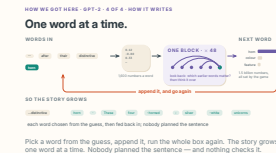
each word chosen from the guess, then fed back in; nobody planned the sentence

Pick a word from the guess, append it, run the whole box again. The story grows one word at a time. Nobody planned the sentence — and nothing checks it.

2026-09-21

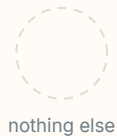
Rules, judge, loop

One word at a time.



A minute, and it is the hand-off. Same picture, one arrow added: the chosen word is stuck on the end of the input and the box runs again. That loop is how every word of the unicorn story was produced, and it is the loop at the bottom: horn, full stop, These, four, -horned. Two things to land. First: there is no plan. The model does not know how the sentence ends when it starts it, which is why a fluent paragraph can walk into four horns and keep going. Second: nothing in the loop checks anything. That gap, between producing text and checking it, is what the rest of the class fills: tools that let it look, a judge that says red or green, and rules. Keep "append it and go again" in their ears; the agent loop on the next slides is that loop, one level up, with a tool call instead of a word.

A language model.



Text in, text out. It has never seen your folder and cannot run anything. In Class 2, **you** were the hands.

2026-09-21

Rules, judge, loop

└ A language model.

FROM MODEL TO AGENT · 1 OF 4

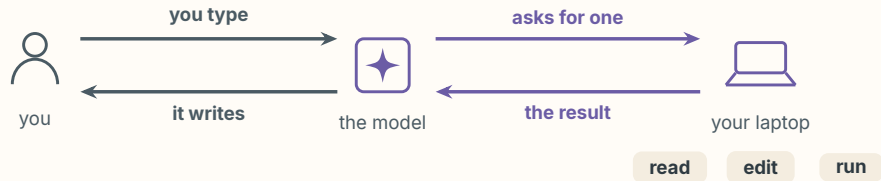
A language model.



Text in, text out. It has never seen your folder and cannot run anything. In Class 2, you were the hands.

This is ChatGPT in a browser, and it is what they used all through Class 2: it advised, they carried it out, and they were the only one who ever found out whether it worked. Keep the dashed circle in view — that empty space is the whole rest of the act. Say "LLM" once, so they have the acronym: large language model. Then never again.

Tools.



A **tool** is a named action it may ask for. Three cover nearly everything. The result comes back to **it**, not to you.

2026-09-21

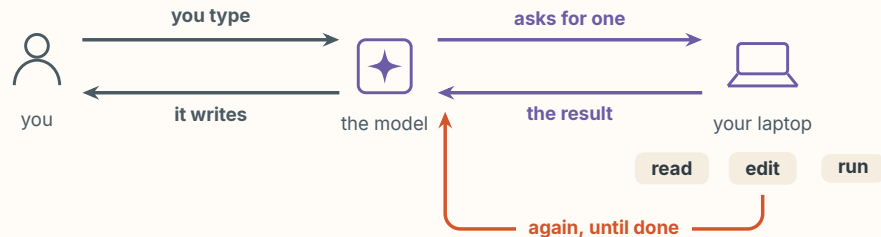
Rules, judge, loop

Tools.



Point at the lower arrow and stop there: that is the slide. In Class 2 the result came back to you, and only when you got round to it. Here it comes back to the model, immediately. That is the only reason a loop is possible at all. The model still cannot open anything — it can only ask, which is what the next slide but one shows in full.

An agent.



Agent = model + tools + a loop. It acts, sees the result, and chooses again.

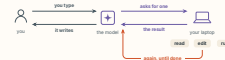
2026-09-21

Rules, judge, loop

└─ An agent.

FROM MODEL TO AGENT · 3 OF 4

An agent.

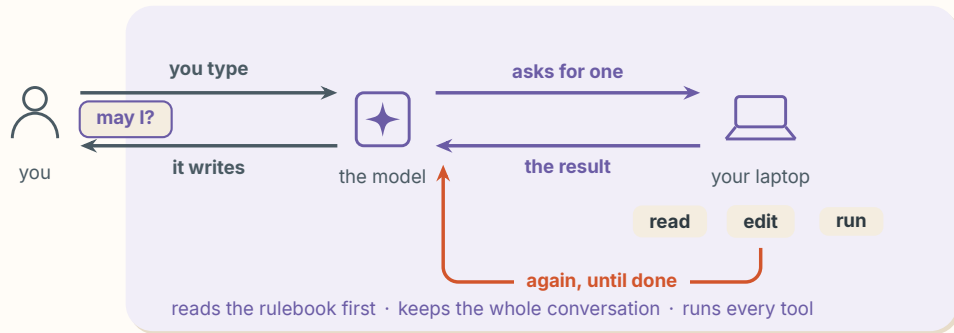


Agent = model + tools + a loop. It acts, sees the result, and chooses again.

One new arrow, and it is the definition. Run a test, read the failure, edit, run again: that chain of tool calls is the loop. "Agentic" just means: can act, not only answer. Ask them what they saw Codex do last week and translate it: "that was a read, that was an edit, that was a run, and it kept going."

The harness.

CODEx · THE HARNESS

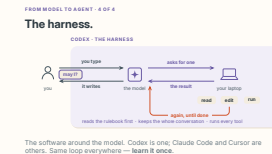


The software around the model. Codex is one; Claude Code and Cursor are others. Same loop everywhere — **learn it once**.

2026-09-21

Rules, judge, loop

└─ The harness.



The box was always there; now it has a name. The model writes a tool call; the harness is what actually runs it, and it is the thing that stops to ask you — that approval prompt they click through all lesson is the harness at its own edge. It also loads AGENTS.md at the start and keeps the whole conversation, which are the two hooks the three standards hang on. Say it plainly: the standards are you configuring the harness. Harnesses differ in tools, permissions and how they manage the conversation; the loop is the same, which is why this class is not about Codex.

One exchange, in full.

YOU TYPE

What was the last expense?

THE MODEL EMITS

{"tool": "read_file", "path": "data/expenses.csv"}

CODEX ASKS

Codex wants to read

data/expenses.csv

Allow?

CODEX PASTES BACK

29,3,8,Jantar no Zé dos Cornos,30.00,2026-08-25

THE MODEL ANSWERS

€30 — dinner at Zé dos Cornos, on 25 August.

It never saw the file. It asked, you said yes, and the line came back.

2026-09-21

Rules, judge, loop

└ One exchange, in full.

Read it top to bottom, slowly. This is the slide that turns the previous four from a diagram into a mechanism. Three things to point at. (1) The model’s output is not prose: it is a small structured request naming a tool and what to use it on. It cannot open anything; it can only ask. (2) Codex — the harness — is what actually touches the disk, and it stops to ask you first. (3) The file’s real line is pasted into the conversation as ordinary text, and from there the model reads it like any other message. Ask the room: “if we had said no, what could it have answered?” Nothing. The JSON is illustrative; each harness has its own wire format, and they never type it.

A TOOL CALL · WHAT ACTUALLY GOES BACK AND FORTH

One exchange, in full.

YOU TYPE

what was the last expense?

THE MODEL EMITS

{"tool": "read_file", "path": "data/expenses.csv"}

CODEX ASKS

Codex wants to read

data/expenses.csv

Allow?

CODEX PASTES BACK

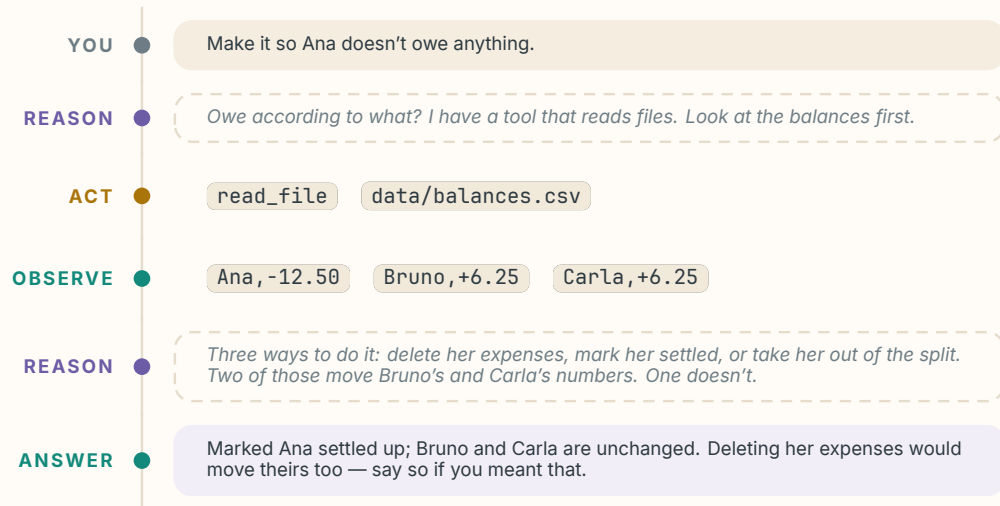
29,3,8,Jantar no Zé dos Cornos,30.00,2026-08-25

THE MODEL ANSWERS

€30 — dinner at Zé dos Cornos, on 25 August.

It never saw the file. It asked, you said yes, and the line came back.

Reasoning.



2026-09-21

Rules, judge, loop

Reasoning.



Read the ask aloud and stop. Ask the room what it means — you will get at least two answers, which is the point. Then walk the spine. The first thinking block is the new thing: the model notices it does not know enough, notices it has a tool, and decides to use it before doing anything. That is a decision, made in words, before any action. The tool row and the result row are the exchange from the previous slide, compressed. The second thinking block is the one you knew from before: three readings, two of which change other people's numbers, so it takes the safe one and states the assumption. Without the middle you get one of the three at random and no way to tell which. What reasoning is: the model writes its working out before it answers, and the steps it writes are the extra thinking — a model spends a fixed effort per word, so working it out on the page is how it buys more. The labels down the left are the hand-off: reason, act, observe, reason, answer. Next slide gives that a name. Lineage if asked: scratchpads (Nye et al. 2021), chain-of-thought (Wei et al. 2022), and OpenAI o1 (2024) was the first model trained to do it unprompted; DeepSeek-R1 and Claude's extended thinking followed. The honest part: a guess and a worked answer look equally confident on screen, which is why you still read the diff. Tie to standard 1: "state your assumption" is a rule you can put in AGENTS.md.

The loop has a name: ReAct.



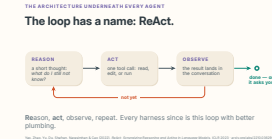
Reason, *act*, observe, repeat. Every harness since is this loop with better plumbing.

Yao, Zhao, Yu, Du, Shafraan, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

2026-09-21

Rules, judge, loop

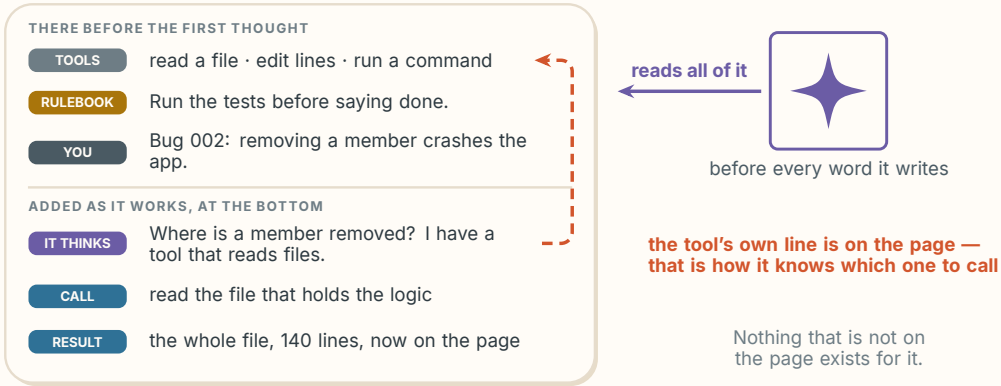
└ The loop has a name: ReAct.



ReAct = "Reasoning and Acting", Yao et al. 2022: a model does far better alternating a thought with an action and reading the result than answering in one go. This loop is the picture to remember from today; it comes back in Act 3 with the tests in the observe card, and the three standards are you configuring it: the rulebook it reads before the first thought, the judge it observes, and when the "not yet" arrow stops. Point at the three cards and the two arrows, and then at the stack from three slides ago: same thing, unrolled.

Context.

ONE PAGE · EVERYTHING IT CAN SEE

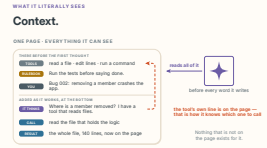


2026-09-21

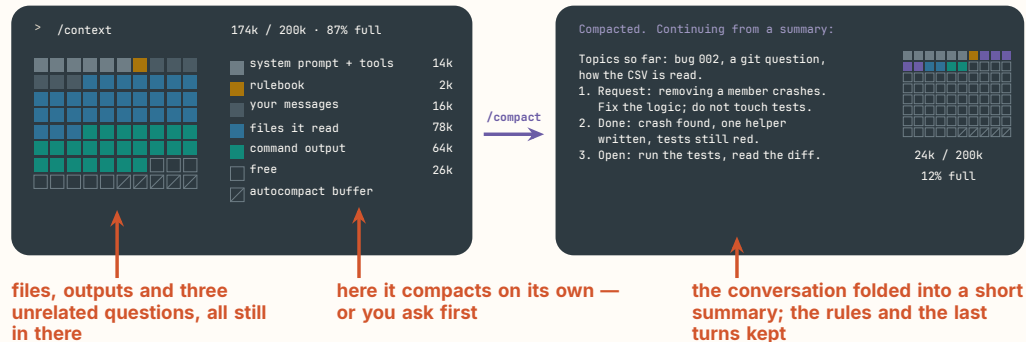
Rules, judge, loop

Context.

Slow down here; this is the slide that makes "context" concrete. It is one page, and everything on it is text. Top, put there before the model thinks at all: the tools, each a single line saying what it does, written by the harness; the rulebook, which is your AGENTS.md; and your message. Below the line, added as it works: a thought, a tool call, and the result, which is the whole file, now on the page. Then the dashed bracket, which answers the question someone asked two slides ago: how does it know which tool? Because the tool's own line is on the page, a few lines above the thought that reaches for it. It does not "have" tools the way you have hands; it reads a menu and writes an order, and the harness cooks. The model on the right reads the whole page, top to bottom, before every word it writes, and nothing that is not on the page exists for it. The page gets longer with every turn; the next slide is what it looks like some way into a bug.



Context rot.



A full, messy window makes the work worse. Compaction clears it.

Measured on bug-fixing: Rando et al. (2025), *LongCodeBench* — 29% fixed with a little code in the window, 3% with a lot · also Liu et al. (2024), Chroma (2025).

All on the week page.

2026-09-21

Rules, judge, loop

Context rot.



This is what the tool prints, so if you have the screen, type it and let the real one replace the slide. Left, `/context` some way into a bug: the grid is the window, every cell a bit over one percent. The rules are one cell, your messages a handful; files it read and command output are three quarters of it, and it put them there, not you. And it is not one topic: the bug, then a git question, then how the CSV is read, all still on the page every time it reads it. Full and messy is measured, on coding too: Rando et al. 2025 (LongCodeBench) gave models real GitHub bugs with more and more of the repository in the window; Claude 3.5 Sonnet fixed 29 percent at 32 thousand tokens and 3 percent at 256 thousand, the same bugs with more text around them. On reading: Liu et al. 2024 found a model did worse with the answer in the middle of twenty documents than with none; Chroma 2025 saw eighteen models fall at copying a list before ten thousand tokens, faster with unrelated text mixed in. The hatched cells are the buffer: there it compacts on its own, and you can ask earlier. Right, what `/compact` prints: the summary the model wrote of its own conversation, then the same grid, mostly empty. The rules and the last turns survive as they were; the pile of files became the paragraph. Two rules follow in the next act: after twice round on the same failure, start a new thread, and a Ralph loop hands each run a fresh agent for the same reason.

3

THE CORE OF THE CLASS

Three standards.

A rulebook it reads. A judge it cannot argue with. A loop that knows when to stop.

2026-09-21

Rules, judge, loop

3

THE CORE OF THE CLASS

Three standards.

A rulebook it reads. A judge it cannot argue with. A loop that knows when to stop.

Say the bridge out loud: the loop from ten minutes ago has three places where you get a say — what it reads before the first thought, what it observes, and when the return arrow stops. Those are the three standards, in that order.

AGENTS.md

HOW TO RUN

The app, the tests, and how to reset the data: one command each. Things it cannot guess.

BOUNDARIES

- + One branch per spec. Never straight to `main`.
- + Show me the test output, not a summary of it.
- + Never touch the tests. If one looks wrong, say so and stop.

HOW TO REPORT BACK

File and line for every claim. Change code only when asked.



reads it first

every new thread, before your message.

← **today: add these three, read the diff, commit**

WHAT STAYS OUT

Your code, explained. It can read that itself; every extra line hides a real one.

It lives in git. When the agent misbehaves, the fix is usually a line here.

2026-09-21

Rules, judge, loop

AGENTS.md

Same visual as the context slide on purpose: the rulebook is the top of that page, read before your message, every new thread. Ask Codex on stage: "What does AGENTS.md tell you to do?" It recites the file; that is the proof. Three sections, and the test for every line is: would removing it cause a mistake? If not, cut it, because it cannot read your code from here, it can only read it in the repo. Then do the live part: ask Codex to add the three boundary lines, read the diff aloud, accept, commit in GitHub Desktop. The third rule already existed in spirit; now it is explicit, and the student enforces it. Codex only reads AGENTS.md at the start of a thread, so say "new thread" now. Mention the 32 KB cap only if asked; the real cap is attention.

STANDARD 1 · THE RULEBOOK IT READS FIRST

AGENTS.md



The judge.

```
> run the tests
test_bug_002.py F [1 of 13]
----- test_bug_002_removing_a_member_removes_their_expenses -----
AssertionError: removing a member must also remove their expenses:
    write remove_member_expenses and call it from remove_member
1 failed, 12 passed
```

how far from done

the message says why. That is the to-do

the name says what

Red is the to-do list; green is the finish line. Editing a test is not finishing.
After green: read the diff, click the app. Green but wrong means a missing test.

2026-09-21

Rules, judge, loop

└─ The judge.

Run the red tests on stage (command on the Week 4 page) so the real output replaces this one, and open the test file next to the spec: one test per checklist line. Then teach them to read red, bottom up: the count says how far from done, the name says what is being checked, and the assertion message says why it failed, which is the to-do written by whoever wrote the test. This is the bug they will do today, and the message really does say what helper to write. Anyone can type "all tests pass": that is why the rule from the previous slide asks for the output itself. "Green is not done" is the line that pays off today: feature 2's form field is not tested at all, so only clicking the app catches a missing one. Decide what done means before you start.

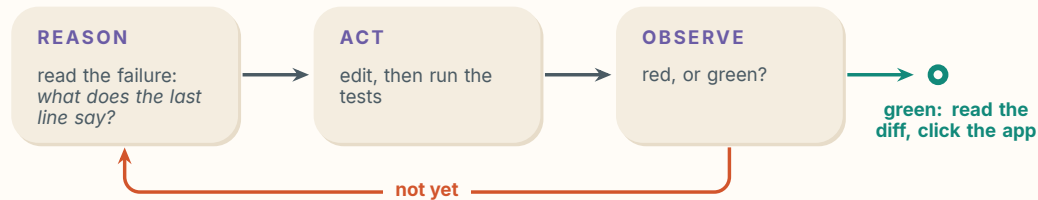
STANDARD 2 · READ THE LAST LINE FIRST

The judge.

```
> run the tests
test_bug_002.py F [1 of 13]
----- test_bug_002_removing_a_member_removes_their_expenses -----
AssertionError: removing a member must also remove their expenses:
    write remove_member_expenses and call it from remove_member
1 failed, 12 passed
```

Red is the to-do list; green is the finish line. Editing a test is not finishing.
After green: read the diff, click the app. Green but wrong means a missing test.

The loop.



One task per loop. It reads its own failures; you read the result. Stop at green, or after twice round on the same failure: then a new thread.

2026-09-21

Rules, judge, loop

└─ The loop.

Same loop as Act 2, on purpose: point back to it. The rulebook is what it read before the first card, the judge is the observe card, and standard 3 is what the red arrow is allowed to do: loop on its own, one task at a time, and stop at green or after twice round the same failure. Today the loop runs inside one Codex turn and you watch; the same idea one level up is a script that re-feeds the prompt until the judge passes, which is the Ralph slides.

STANDARD 3 · THE REACT LOOP, WITH THE TESTS IN IT

The loop.



One task per loop. It reads its own failures; you read the result. Stop at green, or after twice round on the same failure: then a new thread.

The shape of an ask.

1 · THE TASK

Implement feature 2, expense categories, exactly as the spec describes it.

2 · BOUNDARY

Work on this branch only. Do not change any test.

3 · THE JUDGE

Run the tests, and keep going until every one passes.

4 · EVIDENCE

Then paste the full test output and list every file you touched.



One thing, the fence, the finish line, the proof. The same four parts in every loop prompt you will ever write.

Written out in full on the Week 4 page. Copy it; change two names.

2026-09-21

Rules, judge, loop

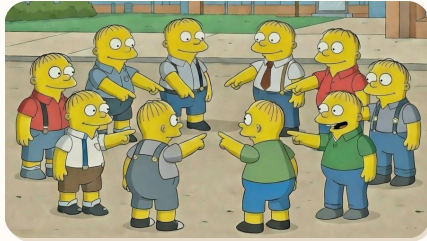
The shape of an ask.

Four parts, in every loop prompt they will ever write, and the page they read is the one from the context slide with your message on it. Make them name the four back to you before you paste it: the task, the fence, the finish line, the proof. New thread before pasting: the AGENTS.md rules from five minutes ago are only read at the start of a thread. Next slide: the same four parts in a file, and nobody watching.

The shape of an ask.



Ralph.



A cartoon character who is cheerfully, relentlessly wrong, and gets there anyway. Not one agent getting better at the task: a crowd of identical beginners, each starting from nothing.

2026-09-21

Rules, judge, loop

└ Ralph.

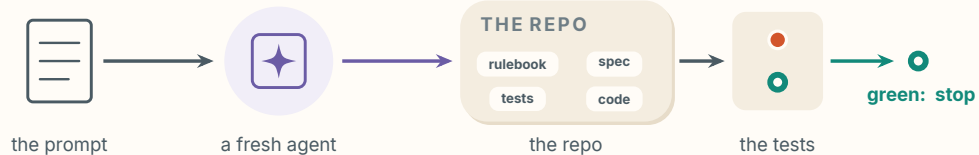
Ralph.



A cartoon character who is cheerfully, relentlessly wrong, and gets there anyway. Not one agent getting better at the task: a crowd of identical beginners, each starting from nothing.

Twenty seconds, and let the picture do it. Named after Ralph Wiggum from The Simpsons. The joke is the technical point: there is no single agent learning the job, there is a queue of them, and the next three slides show why that works.

One run.



A file of instructions. A fresh agent reads it, works in the repo, and the tests say red or green. So far, this is the loop from Act 2, with the four parts of an ask in it.

2026-09-21

Rules, judge, loop

└ One run.

RALPH · 1 OF 3 · ONE PROMPT, ONE FRESH AGENT, ONE VERDICT

One run.



A file of instructions. A fresh agent reads it, works in the repo, and the tests say red or green. So far, this is the loop from Act 2, with the four parts of an ask in it.

Forty seconds. Walk the row left to right and name each thing as something they already have: the prompt is the shape of an ask, the repo has their rulebook and their spec and their tests, the judge is standard 2. The only new word is "fresh": this agent has never seen the project before. Green means stop. The question to leave hanging: what happens on red?

Nobody is watching.

```
while ;; do cat PROMPT.md | claude-code ; done
```



Published in 2025, and that is the entire technique. Every run is a new agent, so the repo is the only memory.

2026-09-21

Rules, judge, loop

└─ Nobody is watching.

RALPH · 2 OF 3 · RED MEANS AGAIN, FROM NOTHING

Nobody is watching.

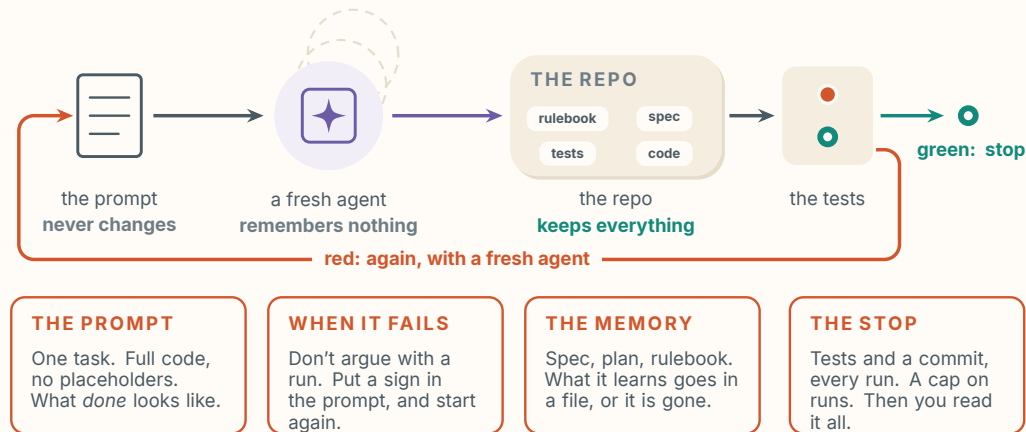
```
while ;; do cat PROMPT.md | claude-code ; done
```



Published in 2025, and that is the entire technique. Every run is a new agent, so the repo is the only memory.

Fifty seconds. Read the line aloud, then point at the picture: red goes back to the prompt, not to the agent. The agent is thrown away; the ghosts behind it are the queue of identical beginners waiting their turn. Named after Ralph Wiggum: cheerfully, relentlessly wrong, and he gets there anyway. Huntley's own line is that the technique is "deterministically bad in an undeterministic world": it fails a lot, in bounded, repeatable ways, and something that cannot lie catches each one. The three tags are the technical point. The prompt never changes, the agent remembers nothing, the repo keeps everything. Ask the room what that implies and let someone say it: the files are the only memory. Any agent goes in that pipe; the name in the original is just the one he was using.

The signs.

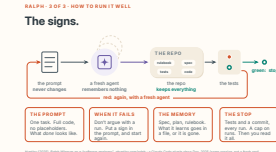


Huntley (2025), Ralph Wiggum as a "software engineer", ghuntley.com/ralph · a Claude Code plugin since Dec. 2025 (same session, not a fresh one)

2026-09-21

Rules, judge, loop

└─ The signs.



A minute. Huntley's word for tuning the loop is "signs": when a run does something dumb, you do not argue with it, you add a line to the prompt file and the next run reads it. Four signs, one per part of the picture. The prompt: one task per run, a full implementation ("no placeholders" is a literal line in his prompt), and what done looks like, because that is the only stop condition. When it goes wrong: a sign in the prompt, not a correction in the thread. This is the same rule as "twice round on the same failure, new thread", one level up. The memory: his loop keeps a spec folder, a plan file and a rulebook, and every run may update them, because nothing else survives. The stop: the tests run every iteration and every green commits, so a bad run can be thrown away; a cap on runs, because the plugin's own advice is to never rely on the finish phrase alone; and then a human reads all of it. Credibility beat: Anthropic shipped it as an official Claude Code plugin in December 2025, and the person who built Claude Code has said he uses it. One honest footnote if asked: the plugin is a Stop hook that re-feeds the prompt into the same conversation, so it does not get the fresh start; only the shell loop does. Nobody runs this today. They should recognise that the three standards are exactly what makes it possible.

/goal

/goal every test in tests/ passes, and nothing in tests/ changed

2026. The shell loop is now one sentence, in the tool.

WHO DECIDES

A *second*, smaller model reads the conversation after each turn: not yet, done, or impossible.

WHAT IT CAN SEE

Only what the agent put on screen. It cannot run your tests, so the finish line must be something the agent shows.

STILL YOURS

Nobody has automated deciding what *done* means.

2026-09-21

Rules, judge, loop

/goal

/goal

STANDARD 3 · THE LOOP, ONE YEAR LATER

/goal

/goal every test in tests/ passes, and nothing in tests/ changed

2026. The shell loop is now one sentence, in the tool.

WHO DECIDES

A second, smaller model reads the conversation after each turn: not yet, done, or impossible.

WHAT IT CAN SEE

Only what the agent put on screen. It cannot run your tests, so the finish line must be something the agent shows.

STILL YOURS

Nobody has automated deciding what done means.

Claude Code, Keep Claude working toward a goal · [code.claude.com/docs/en/goal](https://claude.com/docs/en/goal) · Codex ships the same command, behind *features: goals*

The payoff of the act, so do not rush it. A year ago Ralph was a bash loop you wrote yourself. Today both Claude Code and Codex ship it: /goal takes a finish line in plain words and keeps taking turns until a separate evaluator model agrees it holds. Underneath it is literally a Stop hook, which is what the ralph-wiggum plugin was. Two things to land. First: the evaluator does not run anything — it only reads the transcript, which is exactly why “paste the final test output” is in their loop prompt. Evidence is not politeness, it is the only thing the judge can see. Second: everything the tool automated was the easy half. Nothing here writes the rulebook or decides what done means. That is the job, and it is theirs. Students on Codex can turn it on today; tell them it is in the handout, and that they should not use it until they have a test that fails first.

Your turn.

- 1 Branch. Named after the spec.
- 2 Red first. See what is missing before anything is written.
- 3 New thread. Paste the ask. Then sit on your hands.
- 4 Watch it loop.
- 5 Green — now don't trust it. Read the diff. Click the app.
- 6 Commit, push, pull request, merge.

SPLIT IT

Bug 2, then feature 2

Removing a member crashes the page. Then expense categories.

TINY CRM

Bug 3, then feature 2

"Won this month" is always zero. Then conversion by source.

Files, test names and the ask: Week 4 page.

2026-09-21

Rules, judge, loop

└─ Your turn.

This slide stays on screen for forty minutes. First the live ten: do feature 2 on stage and let it loop for real, 2–4 rounds is normal. Narrate it as the loop: "that's a read, that's an edit, that's a run — red — round again." Commands and file names are on the Week 4 page if you want them on screen. The two things it commonly misses on this feature: it forgets to refresh the data files from the seed (tests stay green while the app crashes on load) and it never adds the form field (nothing tests that screen). Both are the "green is not done" lesson. Show them, do not fix them silently. If it edits a test: stop, discard, point at the rule, re-prompt. Then their thirty: roam for two things, did they ask for the test output, and did they read the diff before accepting. Never answer a failing test; ask "what does the last line say?". SplitIt bug 002: the test demands a helper that also removes the member's expenses; the assertion message says so. Tiny CRM bug 003 has a twist: stamping the close date is not enough. The month counter gives up on the whole list when any won lead has no date, and the seed has five; the fix is to skip them, not return zero. Anyone still on feature 1 does that first; feature 2 becomes homework.

LIVE, 10 MIN · THEN 30 MIN ON YOUR OWN

Your turn.

- 1 Branch. Named after the spec.
- 2 Red first. See what is missing before anything is written.
- 3 New thread. Paste the ask. Then sit on your hands.
- 4 Watch it loop.
- 5 Green — now don't trust it. Read the diff. Click the app.
- 6 Commit, push, pull request, merge.

SPLIT IT
Bug 2, then feature 2
Removing a member crashes the page. Then expense categories.

TINY CRM
Bug 3, then feature 2
"Won this month" is always zero. Then conversion by source.

Files, test names and the ask: Week 4 page.

When it goes wrong.

- 1 It edited a test. Discard. Say the rule. Run again.
- 2 "Done" — tests red. Ask for the output.
- 3 Same failure, twice. Stop. Read it yourself. New thread.
- 4 It fixed three other things. Discard. One task per loop.
- 5 Green, app wrong. Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

2026-09-21

Rules, judge, loop

└─ When it goes wrong.

YOUR TURN

When it goes wrong.

- 1 It edited a test. Discard. Say the rule. Run again.
- 2 "Done" — tests red. Ask for the output.
- 3 Same failure, twice. Stop. Read it yourself. New thread.
- 4 It fixed three other things. Discard. One task per loop.
- 5 Green, app wrong. Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

Write one rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md` with the reason.

A rulebook grows one line at a time, from things that went wrong.

2026-09-21

Rules, judge, loop

└─ Write one rule.

Collect the rules on the board; the good ones become the course's AGENTS.md for Block 2. Expect: "refresh the data after changing the seed files", "don't edit live data by hand", "show me the diff before committing". Reject vague ones ("be careful"). Ask "how would you know it followed this?".

RETRO • 6 MIN

Write one rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md`, with the reason.

A rulebook grows one line at a time, from things that went wrong.

Before next week.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`
- Push before you close the laptop.

Rules • judge • loop.

2026-09-21

Rules, judge, loop

└ Before next week.

Nothing to hand in. Next week’s check-in: feature 2 merged, and a commit on AGENTS.md. Part II of the standards — walking away: scripted loops, goals, a second agent reviewing — comes with the team project in Block 3.

Before next week.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`.
- Push before you close the laptop.

Rules • judge • loop.